

Python Lists & Dictionaries -- Pre-Lab Study Guide

(Beginner-Friendly: No Lambdas, No Comprehensions)

1. What is Indexing?

Think of a list as a row of numbered boxes. Python numbers them starting from 0, not 1. These numbers are called **indices** (singular: index).

```
numbers = [10, 20, 30, 40, 50]
index:    0   1   2   3   4    <-- forward indices (left to right)
index:   -5  -4  -3  -2  -1    <-- backward indices (right to left)
```

```
python

numbers = [10, 20, 30, 40, 50]

# Forward indexing (starts at 0)
first = numbers[0]
print(first)           # Output: 10

third = numbers[2]
print(third)           # Output: 30

# Backward indexing (starts at -1 from the right)
last = numbers[-1]
print(last)            # Output: 50

second_to_last = numbers[-2]
print(second_to_last)  # Output: 40
```

Changing a single element using its index

```
python

numbers = [10, 20, 30, 40, 50]

numbers[0] = 99
print(numbers)          # Output: [99, 20, 30, 40, 50]
```

WARNING -- IndexError

If you use an index number that does not exist, Python crashes with an **IndexError**:

```
python

numbers = [10, 20, 30, 40, 50] # valid indices are 0 to 4

print(numbers[10]) # IndexError: list index out of range
```

2. What is Slicing?

Slicing lets you grab a **chunk** of elements from a list instead of just one.

The format:

```
list[start : stop : step]
```

- **start** -- where to begin (included). Defaults to the very beginning.
- **stop** -- where to end (NOT included). Defaults to the very end.
- **step** -- how many positions to jump. Defaults to 1.

Basic slicing

```
python

numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# From index 2 up to (but NOT including) index 5
result = numbers[2:5]
print(result)      # Output: [2, 3, 4]

# Start missing -- Python begins from index 0
result = numbers[:4]
print(result)      # Output: [0, 1, 2, 3]

# Stop missing -- Python goes all the way to the end
result = numbers[6:]
print(result)      # Output: [6, 7, 8, 9]

# Both missing -- gives a full copy of the list
result = numbers[:]
print(result)      # Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Slicing with a step

```
python

numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Every 2nd element (grab one, skip one)
result = numbers[::2]
print(result)          # Output: [0, 2, 4, 6, 8]

# Every 2nd element, starting from index 1
result = numbers[1::2]
print(result)          # Output: [1, 3, 5, 7, 9]

# Step of -1 means go backwards -- reverses the list
result = numbers[::-1]
print(result)          # Output: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Slicing with negative indices

```
python

numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Last 3 elements
result = numbers[-3:]
print(result)          # Output: [7, 8, 9]

# Everything except the last 3
result = numbers[:-3]
print(result)          # Output: [0, 1, 2, 3, 4, 5, 6]
```

IMPORTANT -- Slicing NEVER crashes

Unlike indexing, slicing silently clips to the valid range:

```
python
```

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Way out of bounds -- returns an empty list, no crash!
result = numbers[100:200]
print(result)          # Output: []

# Negative out of bounds -- clips to the beginning
result = numbers[-100:3]
print(result)          # Output: [0, 1, 2]
```

This is handy, but be careful -- if you expected 5 elements and got 2, Python will NOT warn you. Your index math was probably off.

3. Useful List Techniques

Reverse a list using slicing

```
python

numbers = [1, 2, 3, 4, 5]

reversed_numbers = numbers[::-1]
print(reversed_numbers)    # Output: [5, 4, 3, 2, 1]
print(numbers)             # Output: [1, 2, 3, 4, 5]  -- original unchanged
```

Make a real copy of a list

```
python
```

```
original = [1, 2, 3, 4, 5]

# WRONG -- 'wrong_copy' is just another name for the SAME list
wrong_copy = original
wrong_copy[0] = 99
print(original)      # Output: [99, 2, 3, 4, 5]  -- original changed too!

# CORRECT -- slicing makes a truly separate, independent copy
original = [1, 2, 3, 4, 5]
real_copy = original[:]
real_copy[0] = 99
print(original)      # Output: [1, 2, 3, 4, 5]  -- original untouched
print(real_copy)     # Output: [99, 2, 3, 4, 5]
```

Swap two individual elements

```
python

numbers = [10, 20, 30, 40, 50]

# Swap elements at index 0 and index 4 using a temporary variable
temp      = numbers[0]
numbers[0] = numbers[4]
numbers[4] = temp
print(numbers)      # Output: [50, 20, 30, 40, 10]
```

Insert elements into the middle using slice assignment

```
python

numbers = [1, 2, 5, 6]

# Insert 3 and 4 at index 2 (start == stop, so nothing is removed)
numbers[2:2] = [3, 4]
print(numbers)     # Output: [1, 2, 3, 4, 5, 6]
```

Delete a range of elements

```
python
```

```
numbers = [1, 2, 3, 4, 5, 6]

# Remove elements at indices 2, 3, 4 (not including index 5)
del numbers[2:5]
print(numbers)      # Output: [1, 2, 6]
```

Replace a range with different values (sizes do not have to match)

```
python

numbers = [1, 2, 3, 4, 5]

# Replace 2 elements (at indices 1 and 2) with just 1 element
numbers[1:3] = [99]
print(numbers)      # Output: [1, 99, 4, 5]
```

Rotate a list to the left

```
python

numbers = [1, 2, 3, 4, 5]
k = 2    # rotate left by 2 positions

left_part  = numbers[k:]      # [3, 4, 5]  -- the tail after the cut
right_part = numbers[:k]     # [1, 2]     -- the head before the cut
rotated    = left_part + right_part
print(rotated)               # Output: [3, 4, 5, 1, 2]
```

Chunk a list into smaller pieces

```
python
```

```
def chunk_list(lst, n):
    result = []
    i = 0
    while i < len(lst):
        chunk = lst[i : i + n]      # grab n elements starting at position i
        result.append(chunk)
        i = i + n                  # jump forward by n
    return result

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(chunk_list(numbers, 3))
# Output: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(chunk_list(numbers, 4))
# Output: [[1, 2, 3, 4], [5, 6, 7, 8], [9]]    -- last chunk is shorter
```

WARNING -- The nested list trap

```
python

# WRONG -- all 3 rows are secretly the SAME list object in memory
grid = [[0, 0, 0]] * 3
grid[0][0] = 1
print(grid)      # [[1,0,0], [1,0,0], [1,0,0]]  -- all rows changed!

# CORRECT -- build each row separately so they are truly independent
grid = []
for i in range(3):
    row = [0, 0, 0]
    grid.append(row)

grid[0][0] = 1
print(grid)      # [[1,0,0], [0,0,0], [0,0,0]]  -- only first row changed
```

4. Practice Problems -- Lists

Use this list for all problems unless told otherwise:

```
python

lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

1. Get the first element and the last element using index. Use two different expressions for each (4 total).
 2. Get the last 4 elements using slicing -- without using `len()`.
 3. Reverse `lst` using slicing only.
 4. Get every 3rd element starting from index 1.
 5. Write `split_half(lst)` that splits the list into two halves. It must work correctly when the list length is odd.
 6. Write `swap_halves(lst)` that swaps the first and second half using slice assignment.
 7. Given `lst2 = [0, 1, 2, 3, 4, 5, 6, 7, 8]`, remove the middle 3 elements so the result is `[0, 1, 2, 6, 7, 8]`. Figure out which indices they are first, then use `del`.
 8. Write `rotate_left(lst, k)` that rotates a list left by `k` positions using only slicing (no loop needed).
 9. Write `chunk_list(lst, n)` that returns sublists of size `n` using a `while` loop.
 10. **GA warm-up:** Write `single_point_crossover(parent1, parent2)` -- pick a random cut point and recombine two parents into two children.
-

5. Solutions -- Lists

python

```

import random

# ----- Problem 1 -----
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

# First element (two ways)
print(lst[0])
print(lst[-len(lst)])

# Last element (two ways)
print(lst[-1])
print(lst[len(lst) - 1])

# ----- Problem 2 -----
last_four = lst[-4:]
print(last_four)    # Output: [70, 80, 90, 100]

# ----- Problem 3 -----
reversed_lst = lst[::-1]
print(reversed_lst)

# ----- Problem 4 -----
every_third = lst[1::3]
print(every_third)  # Output: [20, 50, 80]

# ----- Problem 5 -----
def split_half(lst):
    mid = len(lst) // 2    # // is floor division: gives a whole number
    left = lst[:mid]
    right = lst[mid:]      # right side absorbs the extra element when odd
    return left, right

left_half, right_half = split_half(lst)
print(left_half)    # [10, 20, 30, 40, 50]
print(right_half)   # [60, 70, 80, 90, 100]

# ----- Problem 6 -----
def swap_halves(lst):

```

```

mid          = len(lst) // 2
first_half   = lst[:mid]
second_half  = lst[mid:]
lst[:mid]    = second_half
lst[mid:]    = first_half
return lst

numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
print(swap_halves(numbers))
# Output: [60, 70, 80, 90, 100, 10, 20, 30, 40, 50]

# ----- Problem 7 -----
# List:  [0, 1, 2, 3, 4, 5, 6, 7, 8]
# Index: 0 1 2 3 4 5 6 7 8
# The middle 3 are at indices 3, 4, 5
lst2 = [0, 1, 2, 3, 4, 5, 6, 7, 8]
del lst2[3:6]
print(lst2)      # Output: [0, 1, 2, 6, 7, 8]

# ----- Problem 8 -----
def rotate_left(lst, k):
    k          = k % len(lst)    # handles k larger than list length
    left_part  = lst[k:]
    right_part = lst[:k]
    return left_part + right_part

print(rotate_left([1, 2, 3, 4, 5], 2))    # Output: [3, 4, 5, 1, 2]

# ----- Problem 9 -----
def chunk_list(lst, n):
    result = []
    i = 0
    while i < len(lst):
        chunk = lst[i : i + n]
        result.append(chunk)
        i = i + n
    return result

print(chunk_list([1, 2, 3, 4, 5, 6, 7], 3))
# Output: [[1, 2, 3], [4, 5, 6], [7]]

```

```
# ----- Problem 10 -----
def single_point_crossover(parent1, parent2):
    # Pick a random cut point (not at the very start or very end)
    point = random.randint(1, len(parent1) - 1)

    # Child 1 = beginning of parent1 + tail of parent2
    child1 = parent1[:point] + parent2[point:]

    # Child 2 = beginning of parent2 + tail of parent1
    child2 = parent2[:point] + parent1[point:]

    return child1, child2

p1 = [1, 0, 1, 1, 0]
p2 = [0, 1, 0, 0, 1]
child1, child2 = single_point_crossover(p1, p2)
print('Child 1:', child1)
print('Child 2:', child2)
```

6. Why Lists Connect Directly to Your GA Lab

A genetic algorithm represents each solution as a list called a **chromosome**. Every major operation is just a list operation in disguise:

GA Concept	What it actually is in Python
Chromosome	A list of genes, e.g. <code>[1, 0, 1, 1, 0]</code>
Population	A list of chromosomes (list of lists)
Fitness evaluation	Looping through a chromosome and counting correct genes
Single-point crossover	Slicing at one cut point and recombining two parents
Two-point crossover	Slicing at two cut points and swapping the middle section
Mutation	Changing one element: <code>chromosome[i] = new_value</code>
Elitism / selection	Slicing the top few from a sorted population list

7. Genetic Algorithm Skeleton (Step by Step)

python

```

import random

# =====
# STEP 1 -- Create one individual and a full population
# =====

def create_individual(length, gene_pool):
    # Build one random chromosome by picking genes one at a time
    individual = []
    for i in range(length):
        gene = random.choice(gene_pool)
        individual.append(gene)
    return individual

def create_population(pop_size, length, gene_pool):
    # Build a full population by creating individuals one by one
    population = []
    for i in range(pop_size):
        individual = create_individual(length, gene_pool)
        population.append(individual)
    return population

# =====
# STEP 2 -- Fitness function
# =====

def fitness(individual, target):
    # Count how many genes match the target at the same position
    score = 0
    for i in range(len(individual)):
        if individual[i] == target[i]:
            score = score + 1
    return score

# =====
# STEP 3 -- Selection (Tournament style)
# =====

def tournament_selection(population, fitnesses, k=3):

```

```

# Pick k random individuals and return the one with the best fitness
best_individual = None
best_score      = -1

chosen_indices = random.sample(range(len(population)), k)

for index in chosen_indices:
    if fitnesses[index] > best_score:
        best_score      = fitnesses[index]
        best_individual = population[index]

return best_individual

# =====
# STEP 4 -- Crossover
# =====

def single_point_crossover(parent1, parent2):
    # Cut both parents at one random point and swap their tails
    point = random.randint(1, len(parent1) - 1)

    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]

    return child1, child2

def two_point_crossover(parent1, parent2):
    # Cut both parents at two random points and swap the middle section
    points = random.sample(range(1, len(parent1)), 2)
    points.sort()          # make sure the first point is smaller
    pt1 = points[0]
    pt2 = points[1]

    child1 = parent1[:pt1] + parent2[pt1:pt2] + parent1[pt2:]
    child2 = parent2[:pt1] + parent1[pt1:pt2] + parent2[pt2:]

    return child1, child2

# =====
# STEP 5 -- Mutation
# =====

```

```

def mutate(individual, mutation_rate, gene_pool):
    # Go through each gene and randomly replace it
    for i in range(len(individual)):
        chance = random.random()    # float between 0.0 and 1.0
        if chance < mutation_rate:
            individual[i] = random.choice(gene_pool)
    return individual

# =====
# STEP 6 -- Find the best individual in the population
# =====

def find_best(population, target):
    best_individual = population[0]
    best_score      = fitness(population[0], target)

    for i in range(1, len(population)):
        current_score = fitness(population[i], target)
        if current_score > best_score:
            best_score      = current_score
            best_individual = population[i]

    return best_individual

# =====
# STEP 7 -- Score every individual (returns a list of scores)
# =====

def score_population(population, target):
    scores = []
    for individual in population:
        score = fitness(individual, target)
        scores.append(score)
    return scores

# =====
# STEP 8 -- Sort population best-first (no lambda needed)
# =====

def sort_population(population, target):

```

```

pop_copy = population[:]    # work on a copy so original is untouched

for i in range(len(pop_copy)):
    best_index = i
    for j in range(i + 1, len(pop_copy)):
        if fitness(pop_copy[j], target) > fitness(pop_copy[best_index], target):
            best_index = j
    # Swap to bring the best to the front, using a temporary variable
    temp = pop_copy[i]
    pop_copy[i] = pop_copy[best_index]
    pop_copy[best_index] = temp

return pop_copy

# =====
# STEP 9 -- Main GA loop
# =====

def genetic_algorithm(target, pop_size, gene_pool, generations,
                      mutation_rate=0.01, elitism=2):

    length = len(target)
    population = create_population(pop_size, length, gene_pool)

    for gen in range(generations):

        # Score the current population
        fitnesses = score_population(population, target)

        # Elitism: carry the top few directly to the next generation
        ranked_population = sort_population(population, target)
        new_population = ranked_population[:elitism]

        # Fill the rest with children from crossover and mutation
        while len(new_population) < pop_size:
            parent1 = tournament_selection(population, fitnesses)
            parent2 = tournament_selection(population, fitnesses)

            child1, child2 = single_point_crossover(parent1, parent2)

            child1 = mutate(child1, mutation_rate, gene_pool)
            child2 = mutate(child2, mutation_rate, gene_pool)

```

```

        new_population.append(child1)
        new_population.append(child2)

    # Trim back to pop_size in case we added one extra
    population = new_population[:pop_size]

    # Find and print the best individual this generation
    best = find_best(population, target)
    best_score = fitness(best, target)
    print('Generation', gen, '| Best score:', best_score, '| Best:', best)

    # Stop early if the perfect solution is found
    if best_score == length:
        print('Solved at generation', gen, '!')
        break

    return find_best(population, target)

# Run it!
target = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
result = genetic_algorithm(
    target = target,
    pop_size = 50,
    gene_pool = [0, 1],
    generations = 200
)
print('Final answer:', result)

```

8. Dictionaries -- The Basics

A **dictionary** stores data as **key --> value** pairs. Unlike a list (where you access elements by position number), you access a dictionary by **key name**.

```
python
```

```
student = {  
    'name' : 'Lev',  
    'age' : 21,  
    'grade' : 'A'  
}  
  
# Access a value by its key  
name = student['name']  
age = student['age']  
print(name)      # Output: Lev  
print(age)       # Output: 21
```

Crash vs. safe access

```
python  
  
student = {'name': 'Lev', 'age': 21}  
  
# This CRASHES -- 'gpa' key does not exist in the dictionary  
# print(student['gpa'])          # KeyError: 'gpa'  
  
# Safe access using .get() -- returns None instead of crashing  
gpa = student.get('gpa')  
print(gpa)          # Output: None  
  
# Safe access with a default value if the key is missing  
gpa = student.get('gpa', 0.0)  
print(gpa)          # Output: 0.0
```

WARNING -- You CANNOT slice a dictionary

```
python
```

```
d = {'a': 1, 'b': 2, 'c': 3}

# This CRASHES
# print(d[1:3])          # TypeError: unhashable type: 'slice'

# Workaround: convert to a list first, then slice
items      = list(d.items())    # [('a',1), ('b',2), ('c',3)]
first_two  = items[:2]
print(first_two)               # Output: [('a', 1), ('b', 2)]
last_two   = items[-2:]
print(last_two)                # Output: [('b', 2), ('c', 3)]
```

Adding, updating, and deleting entries

```
python

student = {'name': 'Lev', 'age': 21}

# Add a new key
student['grade'] = 'A'
print(student)    # {'name': 'Lev', 'age': 21, 'grade': 'A'}

# Update an existing key
student['age'] = 22
print(student)    # {'name': 'Lev', 'age': 22, 'grade': 'A'}

# Delete a key
del student['grade']
print(student)    # {'name': 'Lev', 'age': 22}
```

Looping through a dictionary

```
python
```

```
d = {'a': 1, 'b': 2, 'c': 3}

# Loop through keys only
for key in d:
    print(key)

# Loop through key-value pairs together
for key, value in d.items():
    print(key, '->', value)

# Loop through values only
for value in d.values():
    print(value)
```

Merging two dictionaries

```
python

d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 3}

# Method 1: Python 3.9 and above (d2 wins on shared keys)
merged = d1 | d2
print(merged)    # {'a': 1, 'b': 99, 'c': 3}

# Method 2: works on all Python 3 versions
merged = {}
merged.update(d1)
merged.update(d2)    # d2 overwrites shared keys
print(merged)        # {'a': 1, 'b': 99, 'c': 3}
```

9. Dictionary Techniques (No Comprehensions)

Invert a dictionary -- swap keys and values

```
python
```

```
def invert_dict(d):
    inverted = {}
    for key, value in d.items():
        inverted[value] = key
    return inverted

original = {'a': 1, 'b': 2, 'c': 3}
flipped = invert_dict(original)
print(flipped)      # Output: {1: 'a', 2: 'b', 3: 'c'}
```

Build a dictionary from two separate lists

python

```
def build_dict_from_lists(keys, values):
    result = {}
    for i in range(len(keys)):
        result[keys[i]] = values[i]
    return result

key_list = ['x', 'y', 'z']
value_list = [10, 20, 30]
d = build_dict_from_lists(key_list, value_list)
print(d)    # Output: {'x': 10, 'y': 20, 'z': 30}
```

Filter a dictionary -- keep entries that pass a condition

python

```
def filter_dict(d, min_value):
    filtered = {}
    for key, value in d.items():
        if value > min_value:
            filtered[key] = value
    return filtered

d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
result = filter_dict(d, 4)
print(result)    # Output: {'a': 5, 'c': 8, 'e': 9}
```

Sort a dictionary by value, largest first

python

```

def sort_dict_by_value(d):
    # Step 1: collect all pairs into a plain list
    items = []
    for key, value in d.items():
        items.append([key, value])

    # Step 2: bubble sort by value, largest first
    for i in range(len(items)):
        for j in range(0, len(items) - i - 1):
            if items[j][1] < items[j + 1][1]:
                # Swap the two pairs using a temporary variable
                temp = items[j]
                items[j] = items[j + 1]
                items[j + 1] = temp

    # Step 3: rebuild the dictionary from the sorted pairs
    sorted_dict = {}
    for pair in items:
        sorted_dict[pair[0]] = pair[1]

    return sorted_dict

d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
print(sort_dict_by_value(d))
# Output: {'e': 9, 'c': 8, 'a': 5, 'b': 2, 'd': 1}

```

Count how many times each item appears in a list

```

python

def count_occurrences(lst):
    counts = {}
    for item in lst:
        if item in counts:
            counts[item] = counts[item] + 1
        else:
            counts[item] = 1
    return counts

data = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
print(count_occurrences(data))
# Output: {'cat': 3, 'dog': 2, 'bird': 1}

```

10. Practice Problems -- Dictionaries

python

```
d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
```

1. Get the value for key `'c'` -- two different ways (one that can crash, one that cannot).
2. Try to access a key `'z'` that does not exist -- once safely, once unsafely.
3. Get the first 2 and last 2 key-value pairs as separate lists.
4. Write `invert_dict(d)` that swaps keys and values using a `for` loop.
5. Write `filter_dict(d, min_val)` that keeps only entries where the value is greater than `min_val`.
6. Write `sort_by_value(d)` that returns a new dict sorted by value, largest first.
7. Given `keys = ['x','y','z']` and `values = [10,20,30]`, build a dict using a `for` loop.
8. Merge `d1 = {'a':1, 'b':2}` and `d2 = {'b':99, 'c':3}` so `d2` wins on shared keys.

Solutions

python

```

# ----- Problem 1 -----
d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}

value1 = d['c']          # direct access
value2 = d.get('c')      # safe access
print(value1)            # 8
print(value2)            # 8

# ----- Problem 2 -----
safe_result = d.get('z') # returns None, no crash
print(safe_result)       # None

safe_with_default = d.get('z', 0)
print(safe_with_default) # 0

# Uncomment to see the crash:
# print(d['z'])           # KeyError: 'z'

# ----- Problem 3 -----
all_items = list(d.items())
print(all_items[:2])     # first 2 pairs
print(all_items[-2:])    # last 2 pairs

# ----- Problem 4 -----
def invert_dict(d):
    inverted = {}
    for key, value in d.items():
        inverted[value] = key
    return inverted

print(invert_dict(d))    # {5:'a', 2:'b', 8:'c', 1:'d', 9:'e'}

# ----- Problem 5 -----
def filter_dict(d, min_val):
    result = {}
    for key, value in d.items():
        if value > min_val:
            result[key] = value
    return result

```

```
print(filter_dict(d, 4))    # {'a': 5, 'c': 8, 'e': 9}
```

```
# ----- Problem 6 -----
```

```
def sort_by_value(d):
    items = []
    for key, value in d.items():
        items.append([key, value])

    for i in range(len(items)):
        for j in range(0, len(items) - i - 1):
            if items[j][1] < items[j + 1][1]:
                # Swap the two pairs using a temporary variable
                temp = items[j]
                items[j] = items[j + 1]
                items[j + 1] = temp

    sorted_dict = {}
    for pair in items:
        sorted_dict[pair[0]] = pair[1]
    return sorted_dict

print(sort_by_value(d))    # {'e':9, 'c':8, 'a':5, 'b':2, 'd':1}
```

```
# ----- Problem 7 -----
```

```
def build_dict_from_lists(keys, values):
    result = {}
    for i in range(len(keys)):
        result[keys[i]] = values[i]
    return result

keys = ['x', 'y', 'z']
values = [10, 20, 30]
print(build_dict_from_lists(keys, values)) # {'x':10, 'y':20, 'z':30}
```

```
# ----- Problem 8 -----
```

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 3}
```

```
merged = {}
merged.update(d1)
```

```
merged.update(d2)    # d2 overwrites 'b'
print(merged)        # {'a': 1, 'b': 99, 'c': 3}
```

11. List + Dictionary Combined -- The GA-Ready Pattern

Once a GA gets more advanced, each individual in the population is stored as a **dictionary** holding its genes and fitness together:

```
python

population = [
    {'genes': [1, 0, 1, 1, 0], 'fitness': 3},
    {'genes': [0, 1, 1, 0, 0], 'fitness': 2},
    {'genes': [1, 1, 1, 1, 1], 'fitness': 5},
]
```

Population is a **list of dicts**, so:

```
python

# Get the second individual (index 1)
second = population[1]
print(second)           # {'genes': [0,1,1,0,0], 'fitness': 2}

# Get its genes
genes = population[1]['genes']
print(genes)            # [0, 1, 1, 0, 0]

# Get a specific gene (e.g. the third gene, index 2)
third_gene = population[1]['genes'][2]
print(third_gene)       # 1
```

Problem 1 -- Extract all gene lists

```
python
```

```
def get_all_genes(population):
    gene_lists = []
    for individual in population:
        gene_lists.append(individual['genes'])
    return gene_lists

genes = get_all_genes(population)
print(genes)
# Output: [[1,0,1,1,0], [0,1,1,0,0], [1,1,1,1,1]]
```

Problem 2 -- Sort population by fitness, best first

python

```
def sort_population_by_fitness(population):
    pop_copy = population[:]    # work on a copy

    for i in range(len(pop_copy)):
        best_index = i
        for j in range(i + 1, len(pop_copy)):
            if pop_copy[j]['fitness'] > pop_copy[best_index]['fitness']:
                best_index = j
        # Swap to bring the best individual to the front
        temp = pop_copy[i]
        pop_copy[i] = pop_copy[best_index]
        pop_copy[best_index] = temp

    return pop_copy

sorted_pop = sort_population_by_fitness(population)
for individual in sorted_pop:
    print(individual)
```

Problem 3 -- Get the top 2 fittest individuals

python

```
def get_top_k(population, k):
    sorted_pop = sort_population_by_fitness(population)
    return sorted_pop[:k]

top_two = get_top_k(population, 2)
for individual in top_two:
    print(individual)
```

Problem 4 -- Compute average fitness

python

```
def average_fitness(population):
    total = 0
    for individual in population:
        total = total + individual['fitness']
    average = total / len(population)
    return average

avg = average_fitness(population)
print(avg)          # Output: 3.333...
```

Problem 5 -- Build a population of dicts from raw gene lists

python

```

def count_ones(genes):
    # Simple fitness function: counts the number of 1s in the genes
    count = 0
    for gene in genes:
        if gene == 1:
            count = count + 1
    return count

def build_population_from_genes(gene_lists):
    population = []
    for genes in gene_lists:
        score = count_ones(genes)
        individual = {'genes': genes, 'fitness': score}
        population.append(individual)
    return population

raw_genes = [
    [1, 0, 1],
    [0, 0, 1],
    [1, 1, 1]
]
pop = build_population_from_genes(raw_genes)
print(pop)
# [{'genes':[1,0,1],'fitness':2}, {'genes':[0,0,1],'fitness':1}, {'genes':[1,1,

```

Problem 6 -- Group individuals by fitness score

python

```

def group_by_fitness(population):
    groups = {}
    for individual in population:
        score = individual['fitness']
        if score not in groups:
            groups[score] = []
        groups[score].append(individual)
    return groups

grouped = group_by_fitness(population)
for score, members in grouped.items():
    print('Fitness', score, '->', members)

```

Problem 7 -- Crossover an entire population of dicts

```
python

import random

def crossover_population(population):
    children = []
    i = 0

    while i < len(population) - 1:
        parent1 = population[i]
        parent2 = population[i + 1]

        genes1 = parent1['genes']
        genes2 = parent2['genes']

        point = random.randint(1, len(genes1) - 1)

        child1_genes = genes1[:point] + genes2[point:]
        child2_genes = genes2[:point] + genes1[point:]

        child1 = {'genes': child1_genes, 'fitness': None}
        child2 = {'genes': child2_genes, 'fitness': None}

        children.append(child1)
        children.append(child2)

        i = i + 2    # move to the next pair

    return children

result = crossover_population(population)
for child in result:
    print(child)
```

12. Quick Reference Cheat Sheet

Lists

```
lst[i]                get one element by index    -- crashes on bad index
(IndexError)
```

<code>lst[i:j]</code>	get a chunk from i to j	-- never crashes
<code>lst[i:j:k]</code>	every kth element from i to j	
<code>lst[::-1]</code>	reverse the entire list	
<code>lst[:]</code>	make an independent shallow copy	
<code>lst[a:b] = [...]</code>	replace a range of elements (lengths can differ)	
<code>del lst[a:b]</code>	remove a range of elements	

Dictionaries

<code>d[key]</code>	get value by key	-- crashes with KeyError if missing
<code>d.get(key)</code>	safe access	-- returns None if missing
<code>d.get(key, default)</code>	safe access	-- returns default if missing
<code>d[1:3]</code>	INVALID	-- TypeError, dicts cannot be sliced
<code>list(d.items())[:n]</code>	workaround: convert to a list, then slice	
<code>d.update(d2)</code>	merge d2 into d	-- d2 wins on shared keys
<code>del d[key]</code>	remove one key-value pair	

List of Dicts (GA pattern)

<code>population[i]</code>	one individual (a dict)
<code>population[i]['genes']</code>	that individual's gene list
<code>population[i]['genes'][j]</code>	one specific gene inside that individual
<code>population[:k]</code>	the top k individuals (after sorting)
<code>population[i]['fitness']</code>	that individual's fitness score